SECURITY TESTING

ASSIGNMENT 3

Tran Dinh Hai - s4041605

Lecturer:
Dr. Sreenivas Tirumala

Table of Contents

1. Exploiting Web Services using SQL Injection Attacks

Q1.1 Exploit the SQL injection vulnerability in the above authentication query to log in to Ted's account without knowing his password, and explain your attack based on your exploited SQL.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_home.php`, is:

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
nickname, Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn -> query($sql);
```

Answer:

Vulnerability analysis: Both values placed into `$sql` originate from the client and are never validated. `$input_uname` is read unmodified from `$_GET['username']`, and `$hashed_pwd` is the SHA-1 digest of `$_GET['Password']`. Hashing the password does not protect this statement: SHA-1 only transforms the *password* field into a fixed-length hexadecimal string, but the vulnerability lies in the neighbouring `name` column, where `$input_uname` is concatenated into the query completely unmodified. Whatever the password field contains, an attacker who controls the `username` field alone can still reshape the statement. Both variables are joined into the query by direct PHP string interpolation, with no parameterised query (an `mysqli` prepared statement with bound placeholders) and no input escaping (`mysqli_real_escape_string()`) applied to either one:

```
WHERE name= '$input_uname' and Password='$hashed_pwd'
```

The single quotes around each variable mark where the developer *intends* the string literal to start and end, but they do nothing to stop the value itself from containing a quote character. Because the query is assembled as plain text before MySQL ever parses it, a quote inside `$input_uname` is indistinguishable from the quote the developer typed around the variable. This is the defining trait of a string-based SQL injection in an authentication check: user input is interpreted as part of the query's *syntax* rather than as inert *data*, so whoever controls the input can rewrite the logic of the query, including removing the password predicate entirely.

Exploiting the vulnerability: I submitted `Ted'#` in the Username field and left the Password field blank, as shown in Figure 1.1. The browser transports the # character in the request line

as the percent-encoded sequence %23; the web server decodes this back to the literal # before PHP ever reads `$_GET['username']`, so the value that reaches `$sql` is the raw character, not the encoded form shown in the address bar.

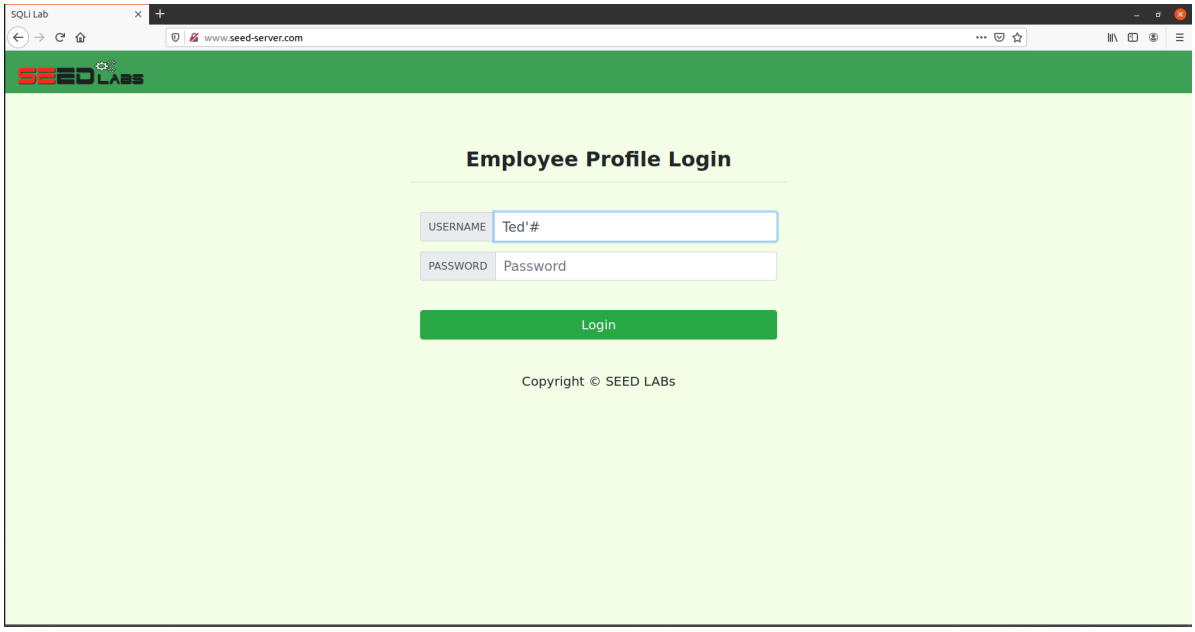


Figure 1.1. The injected username entered with the password field left blank.

With the Password field empty, `$hashed_pwd` is `sha1("")`. Substituting `Ted'#` for `$input_uname` therefore produces the raw statement, in which `$hashed_pwd` appears as its computed SHA-1 digest:

```
1  WHERE name = 'Ted'#' and Password='da39a3ee5e6b4b0d3255bfef95601890afd80709'
```

MySQL parses this left to right. `Ted` selects the intended account name. The apostrophe immediately after it is the one I injected: it closes the string literal that the developer opened with `name= '`, so the database compares `name` against the literal `Ted` only, not `Ted'#`. Parsing then reaches the # character. In MySQL, # begins a single-line comment that extends to the end of the statement, so everything after it – the stray closing quote left over from the original template, the literal `and Password=`, and the SHA-1 hash of the blank password field – is discarded before execution instead of being treated as SQL. The effective condition MySQL actually evaluates is therefore reduced to:

```
1  WHERE name = 'Ted'
```

This selects Ted's row by name alone; no password predicate participates in the query at all. In `unsafe_home.php`, a non-empty `$id` in the returned row is treated as a successful authentication: the script starts a session, stores Ted's identity in it, and calls `drawLayout()` to render his profile. Figure 1.2 confirms this outcome: the application displays the Ted Profile page populated with Employee ID 50000 and Salary 110000, the values recorded for Ted in `sqllab_users.sql`, confirming that the returned row genuinely belongs to Ted rather than to a different or empty account.

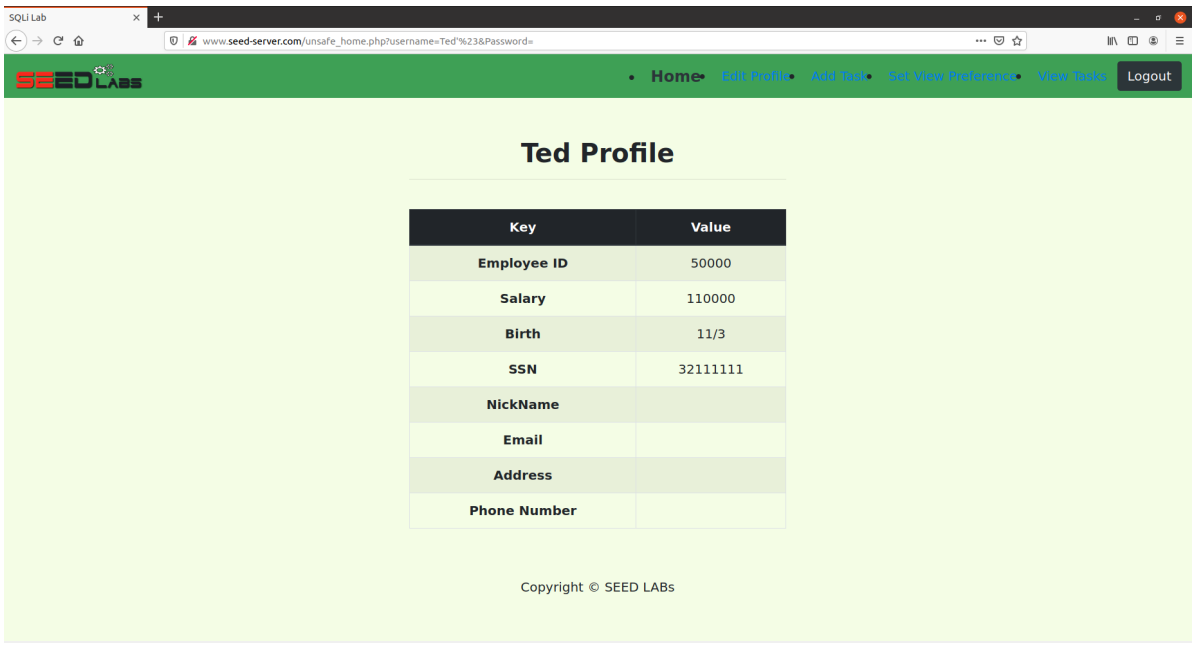


Figure 1.2. Ted's profile displayed after the injected login request.

This demonstrates that the login succeeded without knowledge of Ted's correct password: the SHA-1 comparison that should have enforced it was never evaluated, because the injected comment removed it from the executed query before MySQL had a chance to check it.

Q1.2 Using Ted's session from Q1.1, exploit the same UPDATE statement's SQL injection vulnerability to change Boby's salary to SALARY_1 without knowing either user's password.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_edit_backend.php`, is:

```
$hashed_pwd = sha1($input_pwd);
$sql = "UPDATE credential SET
    nickname='$input_nickname',
    email='$input_email',
    address='$input_address',
    Password='$hashed_pwd',
    PhoneNumber='$input_phonenumber'
WHERE ID=$id;";
$conn->query($sql);
```

Answer:

Vulnerability analysis: `$id` is read from `$_SESSION['id']`, so an attacker cannot set it directly through a request parameter – it is always the ID of whoever is currently logged in, Ted (`ID=5`) in this case. However, `NickName`, `Email`, `Address`, `Password`, and `PhoneNumber` are all read unmodified from `$_GET` and concatenated into `$sql` with no parameterised query and

no escaping, exactly as in `unsafe_home.php`. Because the single quotes around each of these fields only mark the developer's *intended* string boundary, an attacker who breaks out of any one of them can append arbitrary SQL, including a brand-new `WHERE` clause, and comment away the original one. This means the attacker does not need to control `$id` at all: redirecting the `WHERE` clause is enough to make the whole statement apply to any row, regardless of who is logged in.

The full snippet above is only executed when the Password field is non-empty. The actual file wraps it in a condition:

```
if($input_pwd!=''){
    $hashed_pwd = sha1($input_pwd);
    $_SESSION['pwd']=$hashed_pwd;
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',Password='$hashed_pwd',PhoneNumber='$input_phonenumber
    ' where ID=$id;";
}else{
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
}
$conn->query($sql);
```

I left the Password field empty, so the executed statement is built by the `else` branch, which omits the `Password` assignment entirely. This branch is also never checked for success: `$conn->query($sql)` is called and the script unconditionally redirects to `unsafe_home.php` whether the query succeeded or failed, so the application itself gives no on-page indication of the outcome.

Exploiting the vulnerability: While logged in as Ted from Q1.1, I opened Edit Profile and entered the following value in the NickName field, leaving Email, Address, and Phone Number at their pre-filled (blank) values and Password empty, as shown in Figure 1.3:

```
', Salary=1605 WHERE ID=2 --
```

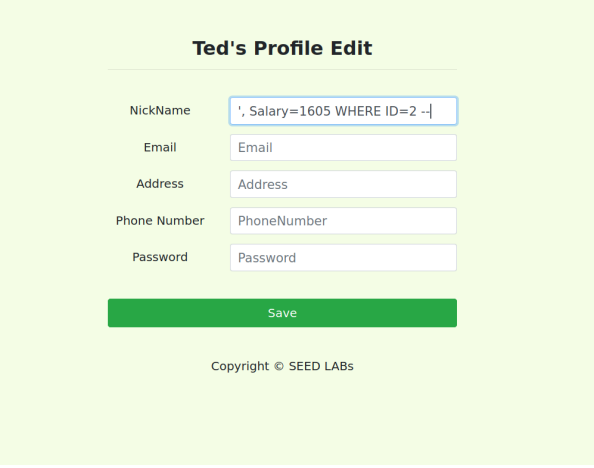


Figure 1.3. The injected NickName payload entered on Ted's Edit Profile form.

Substituting this into the `else` branch's `$sql`, with Ted's session `$id` equal to `5`, produces the raw statement:

```
UPDATE credential SET nickname='', Salary=1605 WHERE ID=2 -- ',email='',address='',
    PhoneNumber='' where ID=5;
```

The leading apostrophe in the payload closes the `nickname='...'` literal immediately, giving `nickname=''` – a harmless no-op, since Boby's NickName is already empty in the seed data. Everything after that quote is now parsed as SQL rather than string content: `, Salary=1605` adds a second column assignment, and `WHERE ID=2` supplies a brand new condition that targets Boby instead of Ted. The trailing `–`, followed by a space, then opens a MySQL end-of-line comment that discards everything after it – the stray closing quote left over from the template, the remaining `email`/`address`/`PhoneNumber` assignments, and the real `where ID=5` clause that would otherwise have kept the update on Ted's own row. The effective statement MySQL actually executes is therefore reduced to:

```
UPDATE credential SET nickname='', Salary=1605 WHERE ID=2
```

This updates Boby's row (`ID=2`) alone; Ted's row is never touched, because the real `where ID=5` clause was commented out before MySQL parsed it.

Since `unsafe_edit_backend.php` gives no visible success or failure message, I verified the result out of band by logging out and logging back in as Boby using the same technique as Q1.1 (`Boby'#` with the Password field left blank), which loads Boby's profile directly from his row in `credential`. Figure 1.4 shows the result:

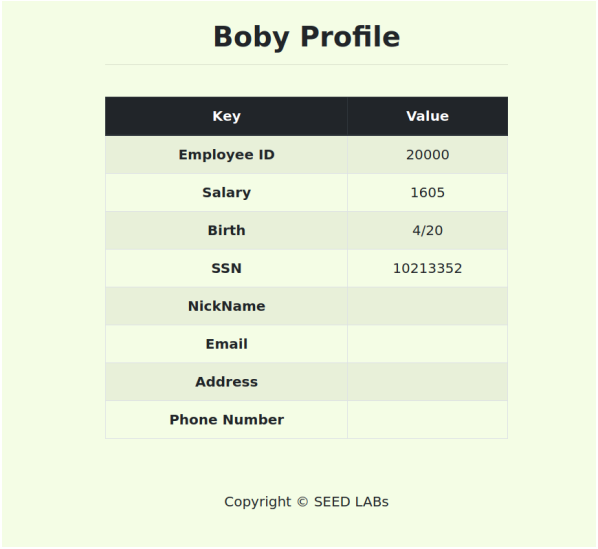


Figure 1.4. Boby's profile after the attack, showing Salary updated to 1605.

Boby's Salary field now reads `1605`, matching SALARY_1 for student s4041605. Employee ID (20000), Birth (4/20), and SSN (10213352) are unchanged from `sqllab_users.sql`, confirming this is genuinely Boby's row, and NickName remains blank, consistent with the injected `nickname=''` assignment being a no-op. This demonstrates that Ted's authenticated session was used to modify a different user's row without knowing Ted's password (already bypassed in

Q1.1) or Boby's password at any point.

Q1.3 Using Ted's session from Q1.1, exploit the same UPDATE statement's SQL injection vulnerability to change Boby's nickname, email, and address to the specified values without knowing either user's password.

Answer:

Vulnerability analysis: This sub-question exploits the same `UPDATE` statement identified in Q1.2, again via the `else` branch of `unsafe_edit_backend.php` that runs when the Password field is left empty:

```
$sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
```

In Q1.2 I broke out of the *first* field (`NickName`) and commented away everything after it, so the earlier, genuine-looking `email`/`address`/`PhoneNumber` values were never applied to any row – only the injected `Salary` assignment mattered. For this sub-question I need three fields (`NickName`, `Email`, `Address`) to actually take effect on Boby's row, so injecting through the first field will not work: anything before the injection point is discarded by the comment, and anything after it never executes on the intended row. Instead, I inject through the *last* field, `PhoneNumber`, immediately before the real `WHERE` clause. A single `UPDATE` statement has exactly one `WHERE` clause governing every `SET` assignment in it, so replacing that clause at the last possible point makes *all* of the preceding assignments – including the legitimate-looking NickName, Email, and Address values – apply to whichever row the injected clause names, rather than to Ted's own row.

Exploiting the vulnerability: While logged in as Ted, I opened Edit Profile and entered the following values, leaving Password empty so the `else` branch above is used, as shown in Figure 1.5:

- NickName: `Tran Dinh Hai`
- Email: `s4041605@rmit.edu.vn`
- Address: `RMIT`
- Phone Number: `' WHERE ID=2#`

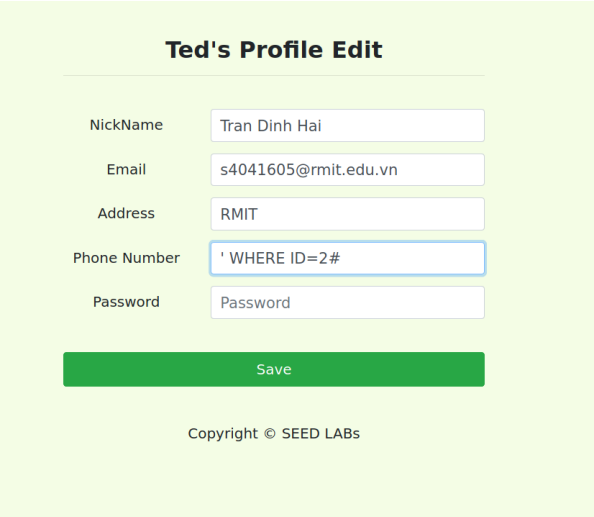


Figure 1.5. The injected values entered on Ted's Edit Profile form, with the payload in Phone Number.

Substituting these into `$sql`, with Ted's session `$id` equal to `5`, produces the raw statement:

```
UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
    ='RMIT',PhoneNumber='' WHERE ID=2#' where ID=5;
```

The apostrophe in the Phone Number payload closes `PhoneNumber='...'` immediately, giving `PhoneNumber=''` – a no-op, since Boby's Phone Number is already blank in the seed data. `WHERE ID=2` then supplies a brand-new condition for the statement, and `#` opens a MySQL comment that discards everything after it: the stray closing quote left over from the template and the real `where ID=5` clause. Because this substitution replaces the statement's only `WHERE` clause rather than being commented out itself, every assignment before it – `nickname`, `email`, and `address` – is retained and now applies to `ID=2`. The effective statement MySQL actually executes is:

```
UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
    ='RMIT',PhoneNumber='' WHERE ID=2
```

I verified the result the same way as Q1.2: logging out and back in as Boby (`Boby'#`, Password blank). Figure 1.6 shows the result:

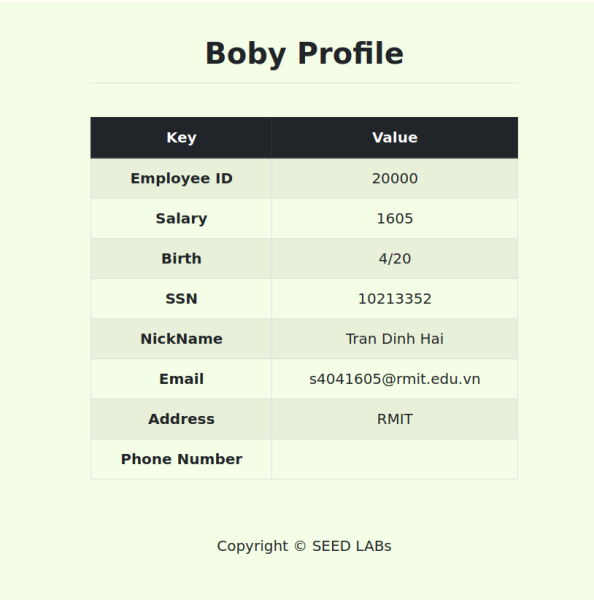


Figure 1.6. Boby's profile after the attack, showing the updated NickName, Email, and Address.

Boby's profile now shows NickName `Tran Dinh Hai`, Email `s4041605@rmit.edu.vn`, and Address `RMIT`, exactly as submitted. Phone Number remains blank, consistent with the injected `PhoneNumber=''` assignment being a no-op. Salary still reads `1605` from Q1.2, and Employee ID (20000), Birth (4/20), and SSN (10213352) remain unchanged, confirming this is still genuinely Boby's row. This demonstrates that Ted's authenticated session was again used to modify Boby's row – this time three fields at once – without knowing Ted's password or Boby's password at any point.

`Q1.4` Identify and explain the vulnerable SQL queries in the two given web pages to display the tasks of the user with the most declared tasks.

The two figures below are given by the assignment brief, consisting of vulnerabilities: the Set View Preference page (Figure 1.7) and the View Tasks page (Figure 1.8).

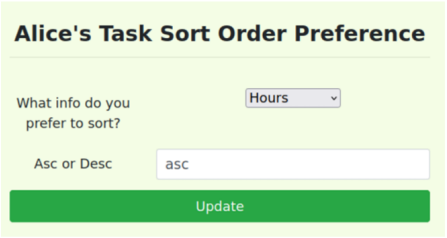
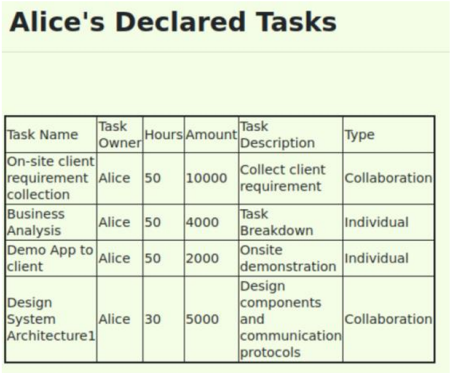


Figure 2 Task Sort Order Preference page of the web application

Figure 1.7. Assignment brief, Figure 2: the Set View Preference page (`unsafe_view_order.php`).



Alice's Declared Tasks

Task Name	Task Owner	Hours	Amount	Task Description	Type
On-site client requirement collection	Alice	50	10000	Collect client requirement	Collaboration
Business Analysis	Alice	50	4000	Task Breakdown	Individual
Demo App to client	Alice	50	2000	Onsite demonstration	Individual
Design System Architecture1	Alice	30	5000	Design components and communication protocols	Collaboration

Figure 3 user's Declared Tasks page of the web application

Figure 1.8. Assignment brief, Figure 3: the View Tasks page (`unsafe_tasks_view.php`).

Answer:

This attack relies on two vulnerable statements working together: one that lets an attacker store arbitrary text unchecked, and one that later reads that text back and executes it as part of a new query.

Vulnerable query 1: writing the sort preference (`unsafe_view_order.php`). In the `unsafe_view_order.php` snippet below, taken from the lab's Docker image at `image_www/Code/`, we can see how the sort preference submitted from Figure 1.7 is written to the database:

```
1  $newfavourite = $_POST[”favourite”] . ” ”;
2  $newfavourite.= $_POST[”order”];
3
4  if($prefid!=””){
5      $sql2 = ”UPDATE preference SET favourite=’$newfavourite’ where PreferenceID=
       $prefid;”;
6  }else{
7      $sql2 = ”INSERT INTO preference(favourite,Owner) VALUES (’$newfavourite’, $id);
       ”;
8  }
9  ...
10 $conn->query($sql2);
```

`$_POST[”favourite”]` and `$_POST[”order”]` are taken directly from the request with no validation – there is no check that `favourite` names a real column (`Name`, `Hours`, `Amount`, `Description`, `Type`) or that `order` is `asc`/`desc`. The two are concatenated into `$newfavourite` and written straight into the `preference` table's `favourite` column via string interpolation, with no escaping. This statement is itself injectable (an attacker could break out of the `’$newfavourite’` literal the same way as in Q1.2/Q1.3), but its more significant role here is that it lets an attacker persist *any* string, unchecked, for later use.

Vulnerable query 2: reading and ordering tasks (`unsafe_tasks_view.php`). In the `unsafe_tasks_view.php` snippet below, also taken from `image_www/Code/`, we can see

```
$sql = "select favourite from preference where owner=$id limit 1";
...
$favourite = $json_a[0]['favourite'];
if($favourite==""){
    $favourite = "hours desc";
}

$sql1 = "select tasks.Name as taskname, credential.Name as ownername, tasks.Hours,
    tasks.Amount,tasks.Description,tasks.Type
from tasks, credential where tasks.owner=credential.ID and tasks.owner=$id "
. "order by tasks." . $favourite ;

$queryList = explode(";",$sql1);

foreach($queryList as $subquery){
    if(strlen($subquery)>0) {
        if (!$tempResult = $conn->query($subquery)) {
            die('There was an error running the query [' . $conn->error . ']\n');
        }
        ...
    }
}
```

The stored `favourite` value is read back from `preference` and concatenated, unquoted and unescaped, directly after the literal text `order by tasks.` to build `$sql1`. Because it is never checked against a whitelist of column names, an attacker who controls its content controls part of the SQL text itself, not just a data value. Worse, the script then calls `explode(";", $sql1)` and executes *every* non-empty resulting fragment as its own independent query via `$conn->query($subquery)` inside the loop. This is a stacked-query mechanism built into the application: if `favourite` contains a `;` followed by a second complete SQL statement, that second statement is executed exactly as if the attacker had submitted it directly.

Why this is exploitable: the complete data flow: An authenticated user (an attacker needs no special privilege beyond a normal session) submits the Set View Preference form with crafted `favourite`/`order` values instead of the intended dropdown selection and `asc`/`desc` text. Query 1 persists this crafted string verbatim into that user's row in `preference`, with no validation. When the same user opens View Tasks, Query 2 reads that string back out of the database and splices it, still unvalidated, into a new `SELECT` immediately after `order by tasks.`. Because the resulting statement is split on `;` and each fragment is executed separately, a `favourite` value of the form `<valid column>; <second SELECT statement>` causes the second statement to run as a genuinely separate query against the same database connection – for example, one that replaces the intended restriction `tasks.owner=$id` with `tasks.owner=userIdMaxTasks()`, so it returns another user's tasks instead of the attacker's own. Query 1 is the injection point (no input validation on write); Query 2 is where the stored payload is later parsed apart and executed (no output validation on read, plus explicit multi-statement execution). Neither vulnerability alone completes the attack – Q1.5 exploits both together.

Q1.5 Exploit the vulnerabilities identified in Q1.4 to make View Tasks display the tasks of the user with the most declared tasks.

Answer:

Step 1: submit the payload: This attack needs both vulnerable statements from Q1.4 to work together. Query 1 is where I plant the malicious text; Query 2 is where that text later gets executed. While logged in as Ted, I opened Set View Preference and filled in the form as shown in Figure 1.9:

- Sort field dropdown (`favourite`): `Hours`
- Asc or Desc field (`order`):

```
;
select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
from tasks, credential
where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
```

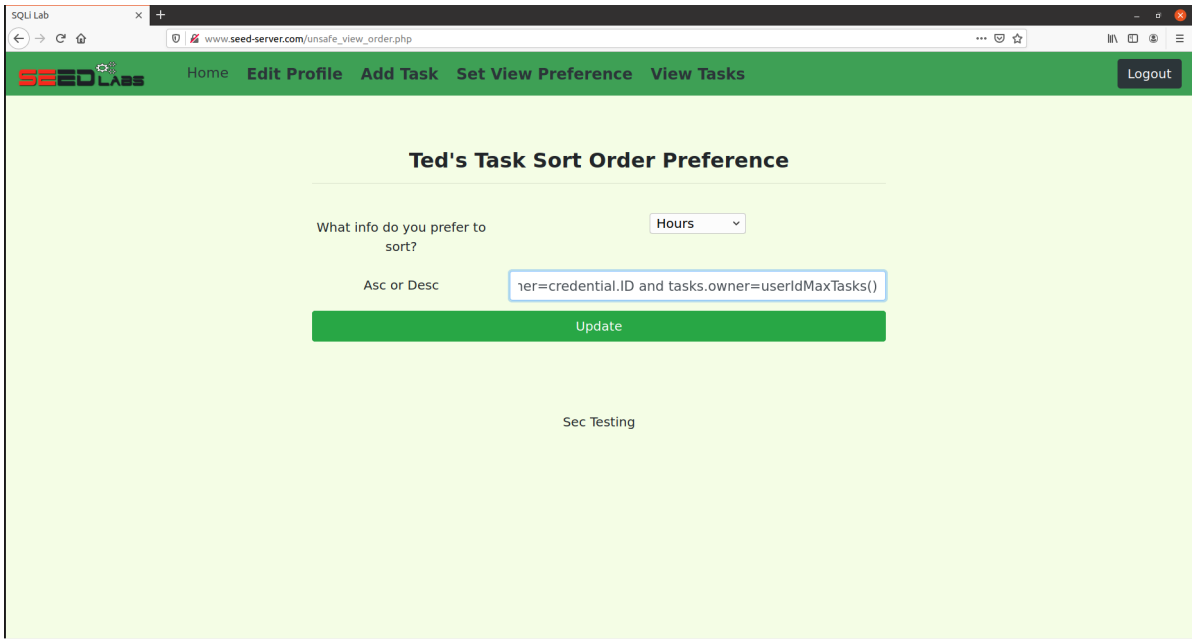


Figure 1.9. The Set View Preference form with `Hours` selected and the injected statement in the Asc or Desc field.

Picking a real column name (`Hours`) for the dropdown keeps the first half of the eventual statement valid SQL. The rest of the attack, a complete second SQL statement, is typed straight into the free-text `order` box. That field has no character restrictions and the form submits it as an ordinary POST request, so no proxy or browser console was needed to send it.

Step 2: how the payload is stored: Query 1 combines the two submitted fields into one string:

```
$newfavourite = $_POST[”favourite”] . ” ”;
```

```
$newfavourite.= $_POST[”order”];
```

With the values above, this builds:

```
Hours ;
select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
from tasks, credential
where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
```

Ted already had a row in `preference`, so this string is written into it unchanged by Query 1's `UPDATE` branch, with no check on its content.

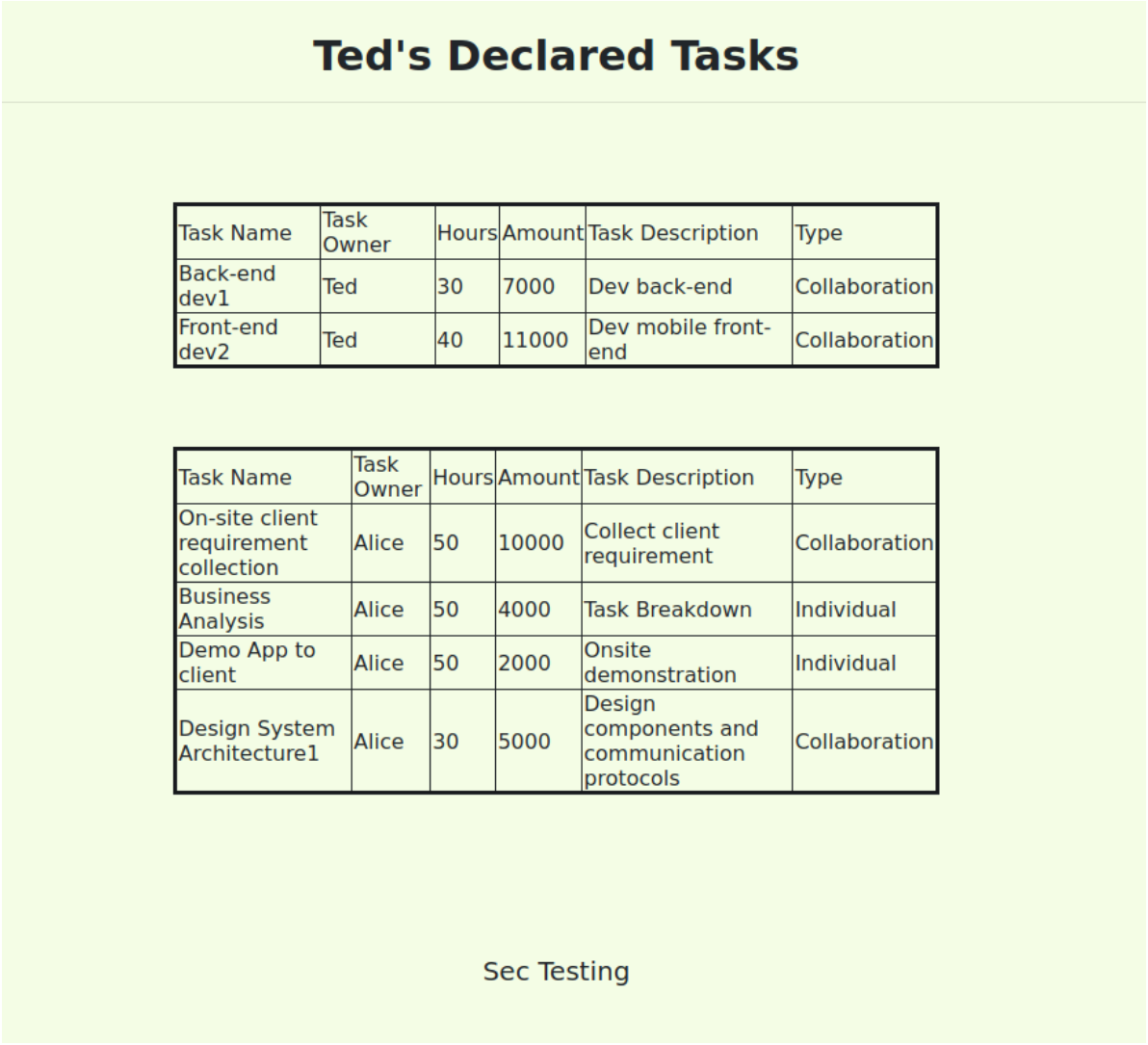
Step 3: how the payload is executed: When View Tasks next loads for Ted, Query 2 reads that same string back out of the database as `$favourite` and appends it to the end of a new query it is building. With Ted's session `$id` equal to `5`, the resulting `$sql1` is (shown here broken across lines for readability; PHP and MySQL both treat it as one continuous string, with no real line breaks):

```
select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
from tasks, credential
where tasks.owner=credential.ID and tasks.owner=5
order by tasks.Hours
;
select tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.
    Amount, tasks.Description, tasks.Type
from tasks, credential
where tasks.owner=credential.ID and tasks.owner=userIdMaxTasks()
```

The semicolon in the middle is the one I typed in the `order` field. Query 2 then runs `explode(”;”, $sql1)`, which cuts this single string into two separate pieces wherever a semicolon appears, and its `foreach` loop sends each non-empty piece to the database as its own, independent query. The table below explains what each of the two resulting queries does:

Query	Restriction on `tasks.owner`	What it returns
1: the original, legitimate half	`tasks.owner=5` – Ted's own session ID, placed there by the application itself	Ted's own declared tasks, sorted by Hours
2: the half I injected	`tasks.owner=userIdMaxTasks()` – a function call I supplied, with no connection to who is logged in	The declared tasks of whichever user currently has the most tasks, joined with `credential` so that user's name is shown too

Step 4: the result: Figure 1.10 shows the View Tasks page after this request. It renders two separate tables, one for each query above. The first table lists Ted's own two tasks (*Back-end dev1*, 30 hours; *Front-end dev2*, 40 hours). The second table lists four tasks, and every one of them has Task Owner `Alice` (*On-site client requirement collection*, *Business Analysis*, *Demo App to client*, *Design System Architecture1*).



Ted's Declared Tasks

Task Name	Task Owner	Hours	Amount	Task Description	Type
Back-end dev1	Ted	30	7000	Dev back-end	Collaboration
Front-end dev2	Ted	40	11000	Dev mobile front-end	Collaboration

Task Name	Task Owner	Hours	Amount	Task Description	Type
On-site client requirement collection	Alice	50	10000	Collect client requirement	Collaboration
Business Analysis	Alice	50	4000	Task Breakdown	Individual
Demo App to client	Alice	50	2000	Onsite demonstration	Individual
Design System Architecture1	Alice	30	5000	Design components and communication protocols	Collaboration

Sec Testing

Figure 1.10. View Tasks after the attack, showing Ted's own tasks and, in the second table, Alice's tasks returned by the injected query.

The second table is the direct output of Query 2's injected half, so `userIdMaxTasks()` resolved to Alice's user ID in the database at the time of this request. This matches `sqllab_users.sql`, where Alice (`ID=1`) is declared four tasks in the original seed data, more than any other user.

Conclusion: An attacker-controlled string, stored with no checks by Query 1, was later cut apart and run as a second, independent SQL statement by Query 2. That second statement ignored the restriction that should have limited results to the logged-in user and instead returned the declared tasks of the user with the most tasks overall, without that user's identity or credentials ever being entered anywhere in the attack. This is the complete attack the question asks for.

2. Exploiting Web Services using Cross-Site Scripting Attacks

Q2.1 Embed a JavaScript program in Samy's "About me" field so that it displays an alert reading "Attack with XSS by" plus your student number when another user views his profile.

Answer:

Vulnerability analysis: Elgg's profile-edit form exposes two separate fields that the assignment's wording of "About me" could plausibly mean: a rich-text **About me** editor and a plain-text **Brief description** field beneath it. I tested the same payload in both; it was stored but never executed in About me, and it executed correctly in Brief description, so this answer targets that field. This lab's Docker image explains why: it modifies two stock Elgg files. Submitted profile input is normally passed through `filter_tags()` in `engine/lib/input.php` before being written to the database, but that function has been overridden to do nothing:

```
function filter_tags($var) {
    // return elgg_trigger_plugin_hook('validate', 'input', null, $var);
    return $var;
}
```

so an unrecognised `<script>` tag is stored exactly as typed instead of being stripped by Elgg's htmLawed sanitiser. On the read side, every page that displays Brief description calls `views/default/output/text.php`, whose encoding call has also been commented out:

```
$value = elgg_extract('value', $vars);
if (!is_scalar($value)) {
    return;
}

//echo htmlspecialchars("{$value}", ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8', false);
echo "{$value}";
```

so the stored string is echoed into the HTML response as live markup instead of being converted to harmless entities by `htmlspecialchars()`. Because neither the write path nor the read path encodes the value, text saved in Brief description reaches every subsequent visitor's browser as executable markup rather than literal text – a stored (persistent) cross-site scripting vulnerability that re-fires for every visitor who loads a page containing it, with no further action needed from the attacker after the initial save.

Exploiting the vulnerability: While logged in as samy, I opened Edit profile and entered the following value in the Brief description field, leaving the About me editor empty, as shown in Figure 2.1:

```
<script>alert('Attack with XSS by s4041605');</script>
```

Saving this submits it to Elgg's profile-save action, which, because `filter_tags()` no longer filters it, writes it unchanged into Samy's description metadata. Every later page that renders Brief description therefore embeds this exact markup verbatim in its HTML response. When a browser parses that response, reaching `<script>` switches it out of normal markup parsing and hands everything up to the matching `</script>` to the JavaScript engine as code rather than displayable text; the engine executes the single statement inside, calling the browser's built-in `alert()` function with the string `'Attack with XSS by s4041605'`, producing a modal dialog. This happens identically regardless of who is viewing the page, since the alert is generated by markup already embedded in the response, not by anything the viewer typed or clicked.

Figure 2.2 shows the alert already firing on Samy's own browser immediately after the "Your profile was successfully saved" confirmation, at `www.seed-server.com/profile/samy`, confirming execution happens automatically the first time the profile page is rendered. Figure 2.3 shows the same alert firing in a separate session logged in as Alice, triggered simply by opening the site's Members listing page rather than Samy's profile page directly; Elgg's Members list renders each user's Brief description as part of their card, so the unescaped script executes there too. This demonstrates that a script stored once in Samy's profile executes automatically for a different user, Alice, on any page that surfaces the field, without her submitting any input of her own.

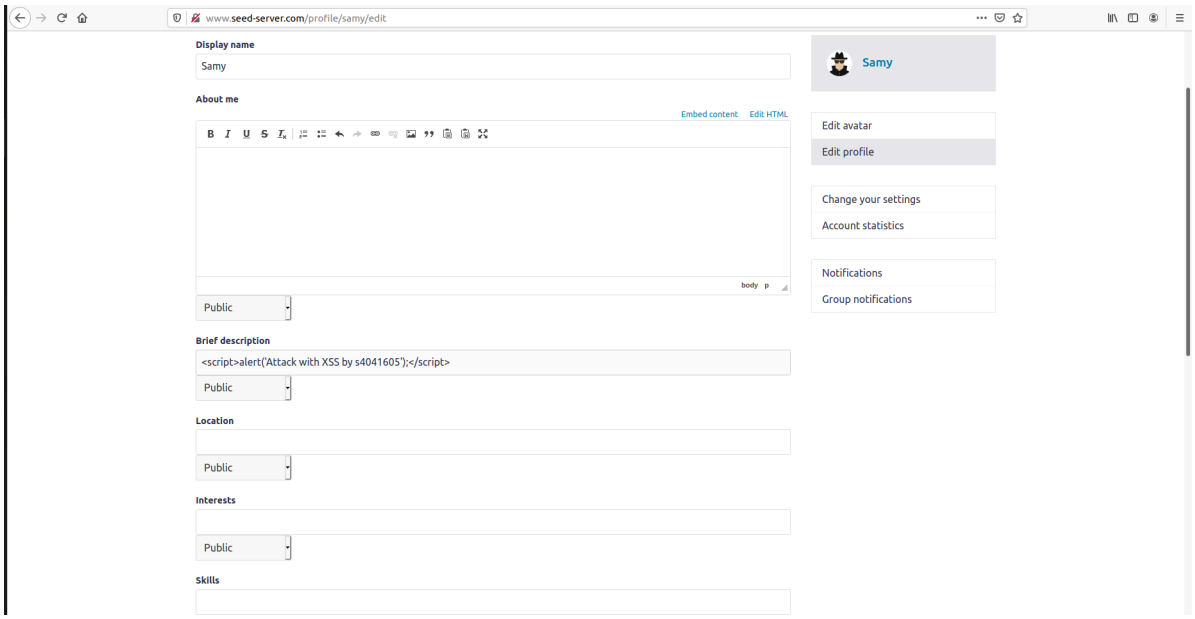


Figure 2.1. Samy's edit-profile page showing the saved payload in the Brief description field.

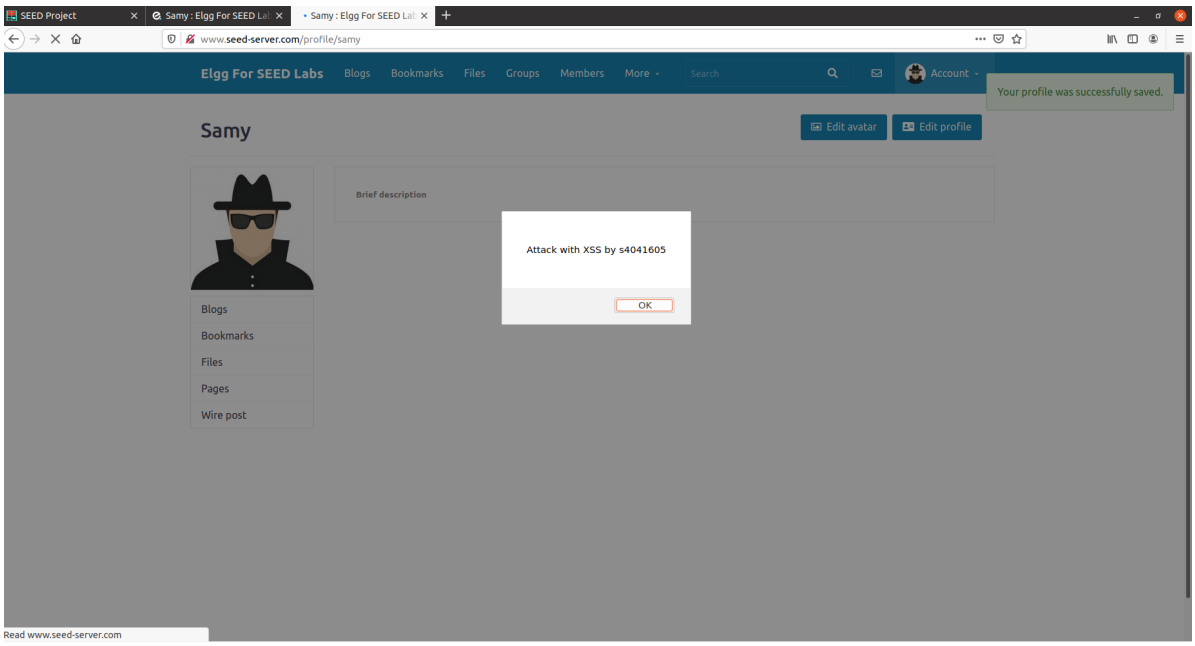


Figure 2.2. The alert firing on Samy's own profile page immediately after saving the payload.

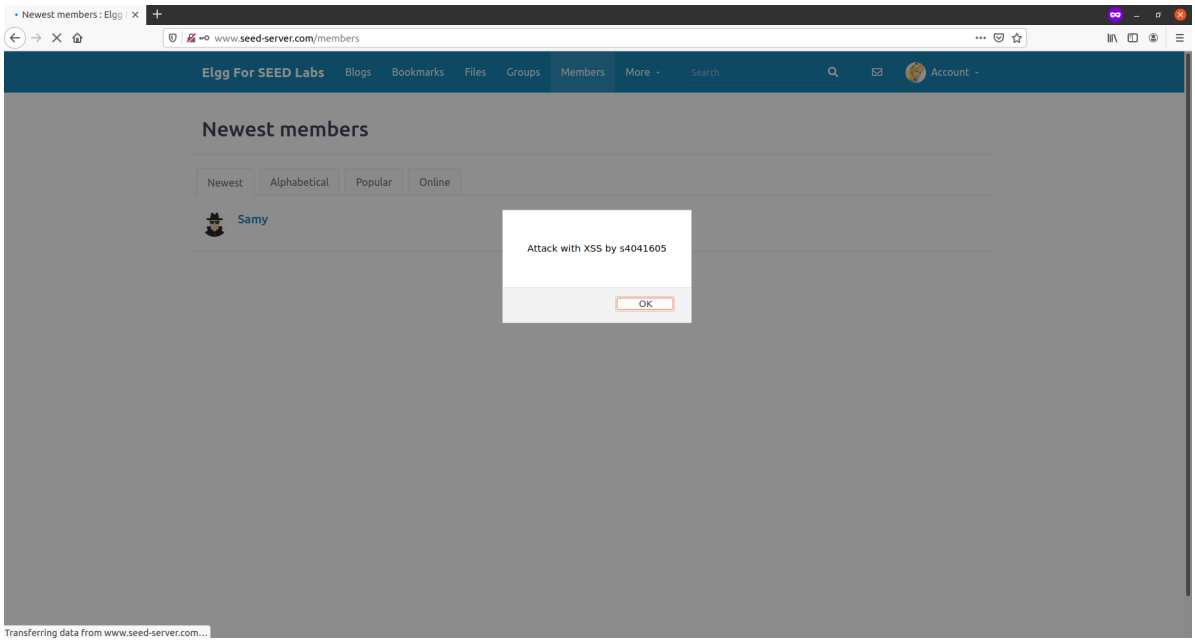


Figure 2.3. The same alert firing in Alice's session after opening the Members listing page, which also renders Samy's Brief description.

Q2.2 Embed a JavaScript program in Samy's "About me" field so that it displays the viewing user's cookies in an alert when another user views his profile.

Answer:

Vulnerability analysis: This sub-question exploits the same missing controls identified in Q2.1: `filter_tags()` in `engine/lib/input.php` no longer sanitises submitted profile input before it is stored, and `views/default/output/text.php` echoes the stored Brief description value with its `htmlspecialchars()` call commented out. Q2.1 already established that a `<script>` tag placed in that field is stored verbatim and re-executed, unmodified, in every visitor's browser on any page that renders it. This sub-question uses that same execution primitive with a different statement inside the tag: instead of a fixed string, `document.cookie` reads the cookies the victim's own browser is currently holding for `www.seed-server.com`, and passes that string into `alert()`. Because the payload runs inside the victim's browser, in the victim's own origin, it has exactly the same access to `document.cookie` that any legitimate script on the site would have – the vulnerability is not in how cookies are read, but in the fact that Elgg lets an attacker plant arbitrary script into a page the victim's browser will trust and execute.

Exploiting the vulnerability: While logged in as samy, I replaced the Brief description payload from Q2.1 with:

```
1  <script>alert(document.cookie);</script>
```

as shown in Figure 2.4. Saving this stores it unchanged in Samy's description metadata, for the same reason established in Q2.1. Figure 2.5 shows the result on Samy's own browser immediately after saving, at `www.seed-server.com/profile/samy`: the alert reads `Elgg=9ke165c5mm3lm631nm1cs87hid`, Samy's own session cookie. Figure 2.6 shows the same payload firing in a separate session logged in as Alice, triggered by opening the Members listing page; the alert here reads a different value, `Elgg=9tdgj0mhjh168ktht35ivlgt2l`. The two alerts showing two different cookie values from two different logged-in sessions confirms that each victim's own browser executed `document.cookie` in its own security context and exposed its own session cookie, not Samy's.

Impact of obtaining the cookie: `Elgg` is Elgg's session cookie: the server treats whoever presents a given session's cookie value as the corresponding logged-in user, without checking the password again. The alert box used here only proves the value is readable; it is not the only way to get it out. The identical injected statement could instead send `document.cookie` to a server the attacker controls, for example by setting it as the source of an off-site image tag or through a background `fetch()` request, which would exfiltrate the value silently, with no dialog box and no indication to the victim that anything happened. Once an attacker holds a captured value such as Alice's `Elgg=9tdgj0mhjh168ktht35ivlgt2l`, setting that same string as their own browser's cookie for `www.seed-server.com` and reloading the site would present Elgg's server with a session identifier it already recognises as Alice's, granting access to her account – reading her private data, changing her profile, or acting as her – without the attacker ever learning her password. This attack works here specifically because Alice's session cookie was readable by `document.cookie` in the first place, meaning it was not marked with the `HttpOnly` attribute that would otherwise have hidden it from JavaScript entirely.

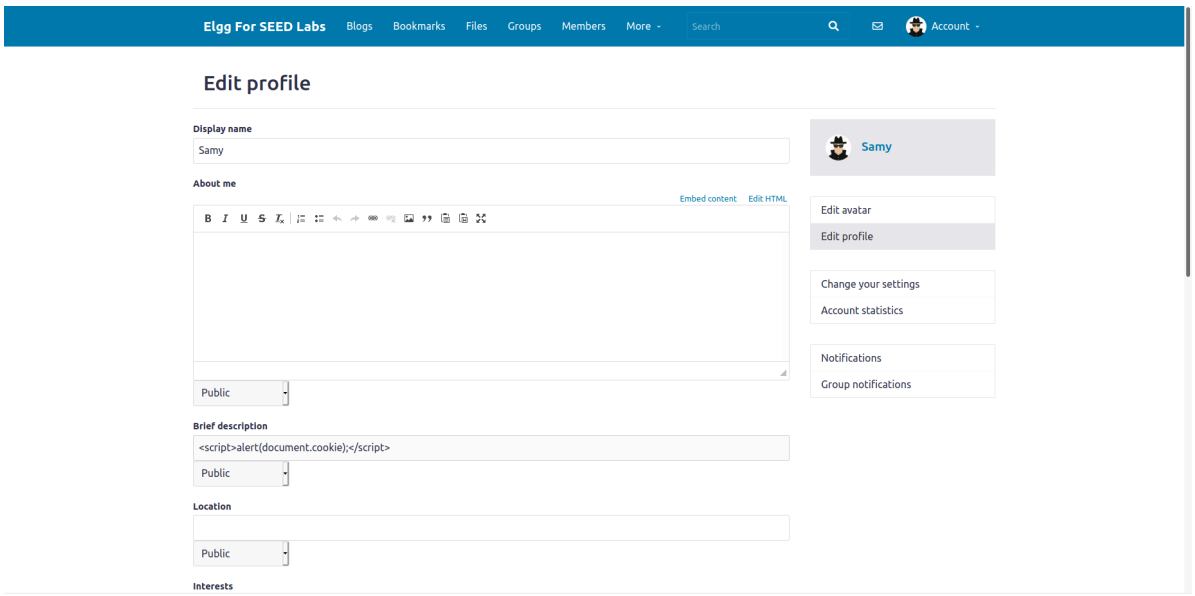


Figure 2.4. Samy's edit-profile page showing the cookie-stealing payload saved in the Brief description field.

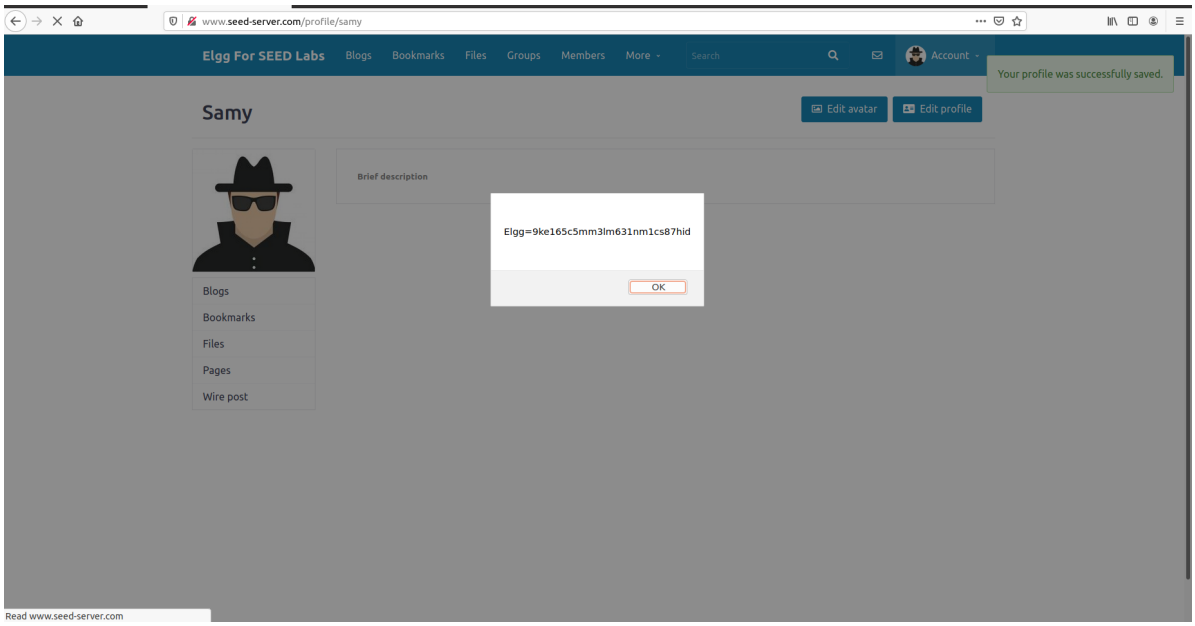


Figure 2.5. The alert showing Samy's own session cookie immediately after saving the payload.

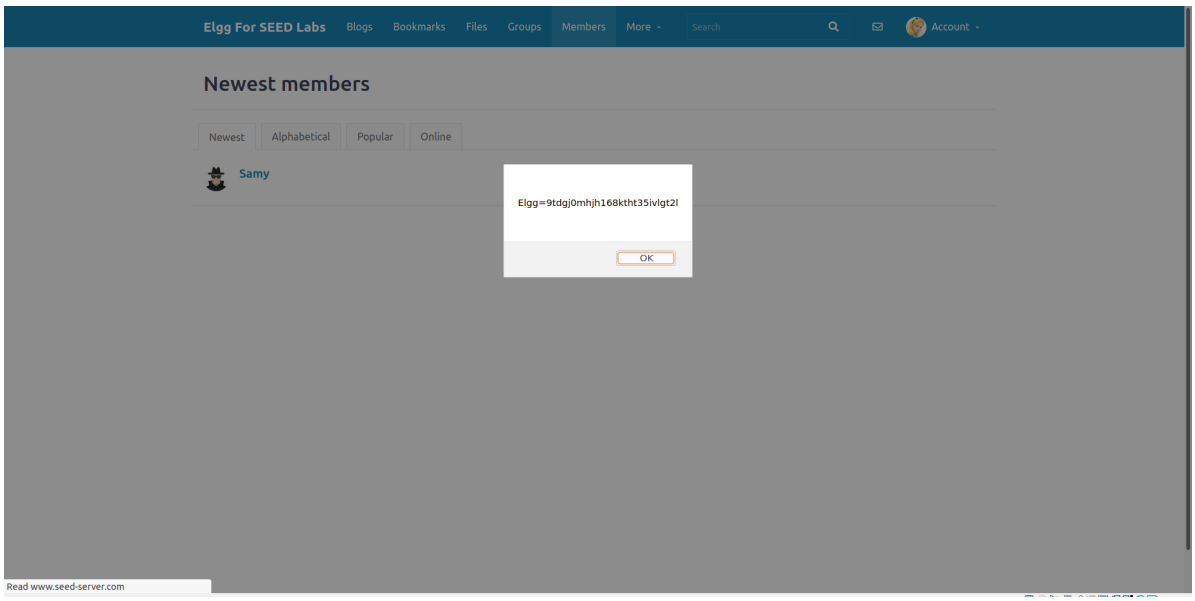


Figure 2.6. The same payload firing in Alice's session, showing a different cookie value belonging to Alice.

Q2.3 Using Firefox's HTTP inspection tools on a legitimate profile edit, determine the three missing values needed to complete the supplied skeleton script that forges a profile-edit request from the victim's browser.

The assignment brief supplies the following skeleton JavaScript for the worm, leaving three "FILL-IN" fields for the values a legitimate profile-edit request actually uses:

```
<script type='text/javascript'>
window.onload=function(){
    //JavaScript code to access user name, user guid, Time Stamp
    //__elgg_ts and Security Token __elgg_token
    var userName=elgg.session.user.name;
    var guid='&guid='+elgg.session.user.guid;
    var ts='&__elgg_ts='+elgg.security.token.__elgg_ts;
    var token='&__elgg_token='+elgg.security.token.__elgg_token;
    var desc='&description= *** '; //FILL-IN
    var name='&name='+userName;
    //Construct the content of your url.
    var sendurl='***'; //FILL-IN
    var content=token+ts+name+desc+guid;
    var samyGuid=***;     //FILL-IN
    if(elgg.session.user.guid!=samyGuid) {
          //Create and send Ajax request to modify profile
          var Ajax=null;
          Ajax=new XMLHttpRequest();
          Ajax.open('POST',sendurl,true)
          Ajax.setRequestHeader('Host','www.seed-server.com');
          Ajax.setRequestHeader('Content-Type',
                                'application/x-www-form-urlencoded');
          Ajax.send(content);
    }
```

```
25  }
26  </script>
```

Answer:

Finding the required values: While logged in as samy, I opened Firefox's Network tab, made a small edit on `profile/samy/edit` (typing `hello` into the About me editor), and saved it to capture the resulting POST request. Figure 2.7 shows the Headers panel for that request: it was sent to `http://www.seed-server.com/action/profile/edit`, which supplies the `sendurl` value. Figures 2.8 and 2.9 show the request body, a `multipart/form-data` submission listing every profile field by name. Two details from this body matter for completing the skeleton. First, the field named `guid` is highlighted in Figure 2.9 with value 59; since this request was Samy editing his own profile, 59 is Samy's own GUID, which supplies `samyGuid`. Second, the field actually named `description` in this Elgg build (visible in Figure 2.8 holding `<p>hello</p>`, the text I typed) is the rich-text **About me** editor, not the **Brief description** field used in Q2.1 and Q2.2 – that field is submitted separately as `briefdescription`. The skeleton's `desc` variable writes into `description`, so it targets About me specifically, and the assignment brief already specifies this value directly as the student's RMIT number.

One more difference is visible in Figure 2.7: the browser sent this request as `multipart/form-data`, while the skeleton's own `Ajax.setRequestHeader` call declares `application/x-www-form-urlencoded`. This does not stop the forged request from working, because Elgg's PHP action handler reads submitted values from `$_POST` regardless of which encoding produced them, provided the `Content-Type` header correctly matches the body format actually sent – which it does here, since the skeleton builds `content` as a plain `key=value&key=value` string and labels it accordingly, rather than reproducing the browser's multipart framing.

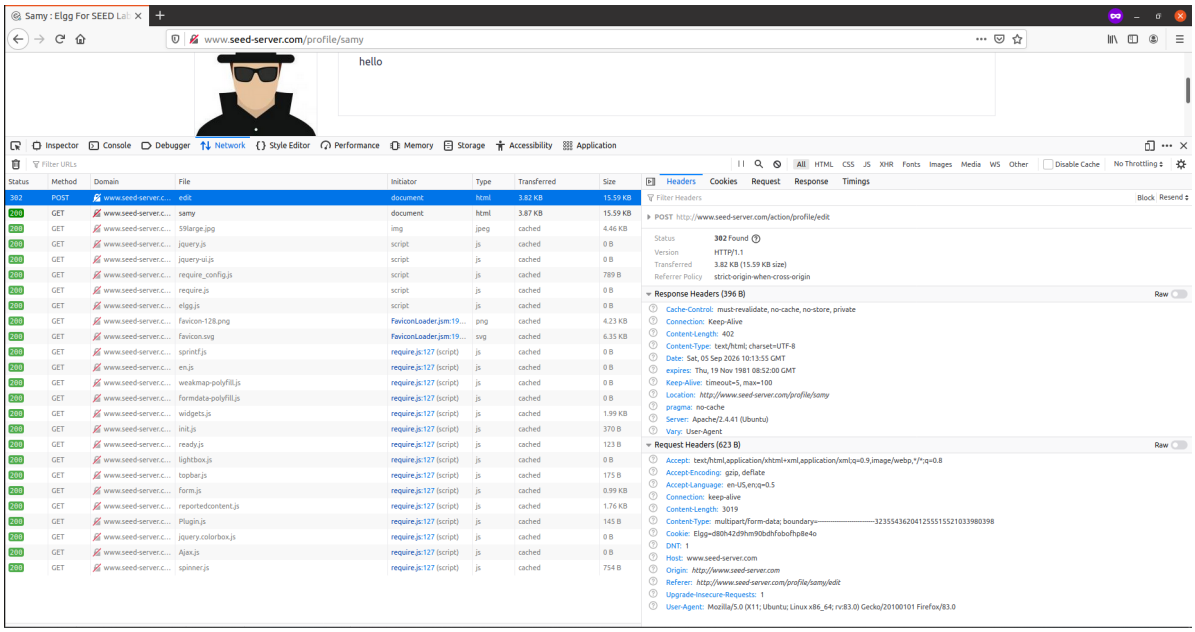


Figure 2.7. The Headers panel for the captured profile-edit request, showing the target URL.

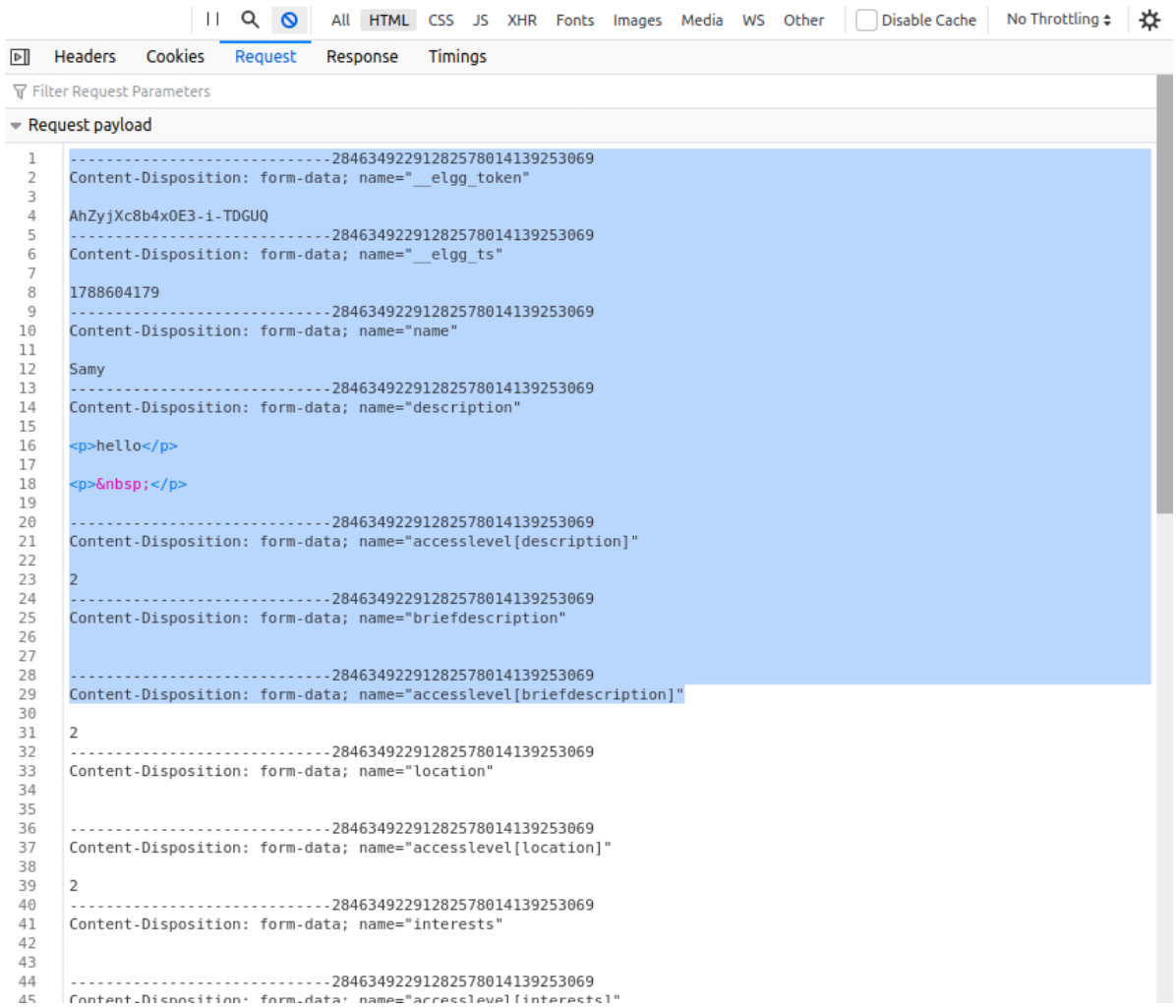


Figure 2.8. The start of the request body, showing the security token, timestamp, name, and the `description` (About me) field.

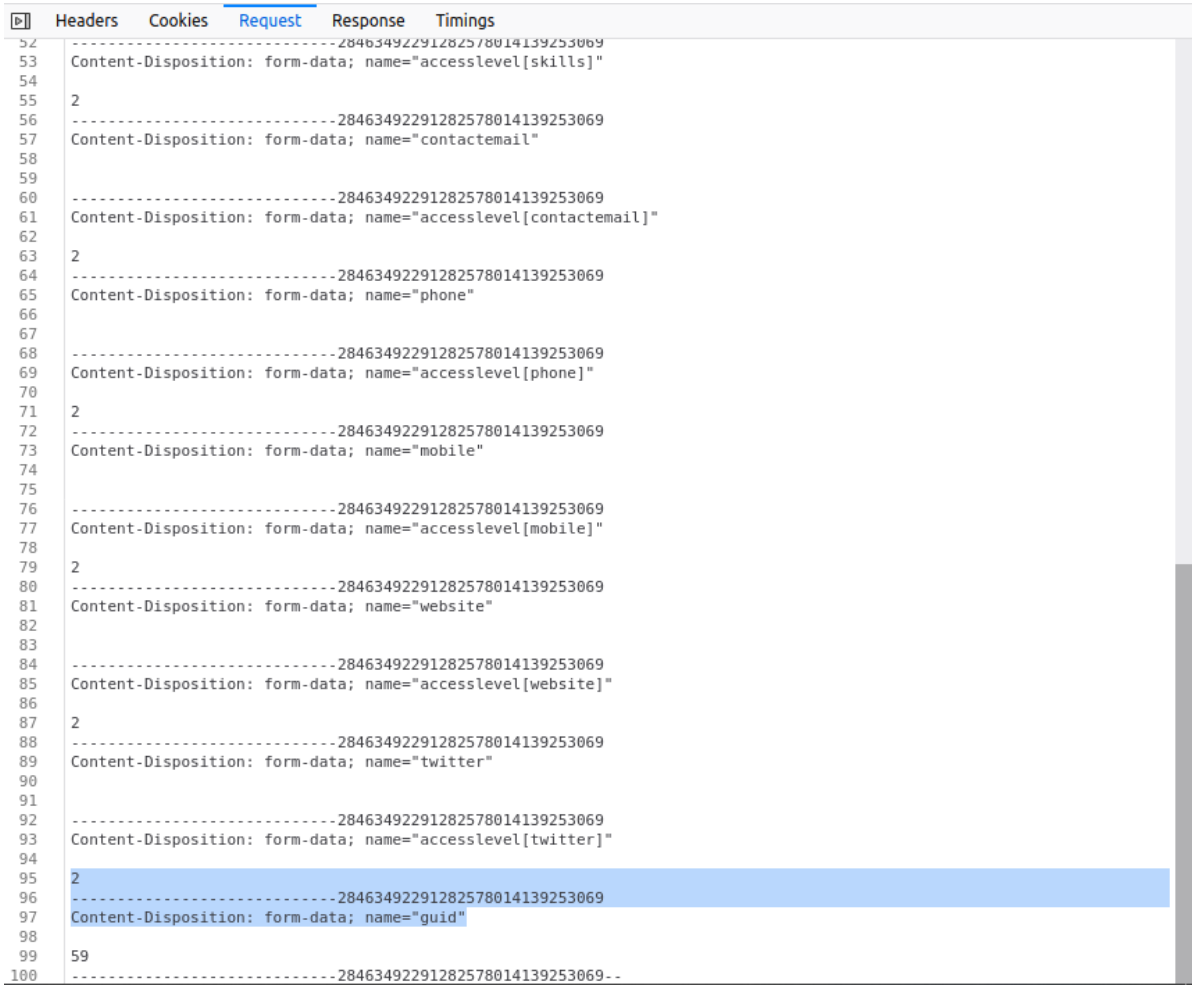


Figure 2.9. The end of the request body, with the `guid` field highlighted.

Completed script: Substituting `s4041605` for `desc`, the captured endpoint for `sendurl`, and `59` for `samyGuid` gives:

```
<script type='text/javascript'>
window.onload=function(){
    //JavaScript code to access user name, user guid, Time Stamp
    //__elgg_ts and Security Token __elgg_token
    var userName=elgg.session.user.name;
    var guid='&guid='+elgg.session.user.guid;
    var ts='&__elgg_ts='+elgg.security.token.__elgg_ts;
    var token='&__elgg_token='+elgg.security.token.__elgg_token;
    var desc='&description=s4041605';
    var name='&name='+userName;
    //Construct the content of your url.
    var sendurl='http://www.seed-server.com/action/profile/edit';
    var content=token+ts+name+desc+guid;
    var samyGuid=59;
    if(elgg.session.user.guid!=samyGuid) {
          //Create and send Ajax request to modify profile
          var Ajax=null;
```

```
            Ajax=new XMLHttpRequest();
            Ajax.open('POST',sendurl,true)
            Ajax.setRequestHeader('Host','www.seed-server.com');
            Ajax.setRequestHeader('Content-Type',
                                  'application/x-www-form-urlencoded');
            Ajax.send(content);
        }
    }
    </script>
```

The four remaining pieces of `content` – `token`, `ts`, `name`, and `guid` – are read dynamically from `elgg.session` and `elgg.security.token` at the moment the script runs in the victim's browser, so they always carry the viewing victim's own valid security token, timestamp, and GUID rather than Samy's, which is what allows this request to pass as the victim's own authenticated action when it later runs inside Alice's session in Q2.4. The `if` guard compares the viewer's GUID to Samy's fixed GUID (59) so that the script does not try to overwrite Samy's own profile when Samy himself happens to load the page.

Q2.4 Store the completed Q2.3 script in Samy's "About me" field so that it automatically injects your student number into another user's profile when they view his page.

Answer:

Vulnerability analysis: This sub-question combines the two vulnerabilities already established. Q2.1 showed that `filter_tags()` and `output/text.php` let an attacker plant a `<script>` tag in Samy's Brief description that executes, unmodified, in every visitor's browser. Q2.3 showed that Elgg's profile-edit action can be triggered by any script running on the site, because the security token, timestamp, and GUID it requires are all readable from `elgg.session` and `elgg.security.token` in the current page. As with Q2.2's assignment's stated field, the question again refers to "About me", but the completed worm from Q2.3 must itself be planted in **Brief description** to execute at all, for the same reason established in Q2.1; the field it writes into when it runs is **About me**, since that is what the `description` parameter in the forged request updates. What makes this a worm rather than just a self-contained alert is that the forged request needs no value stolen from anywhere: because the injected script runs inside the victim's own authenticated browser session, it already has direct access to that victim's own live token and GUID, the same values Elgg itself would supply if the victim clicked Save. The server therefore has no way to distinguish this request from a genuine one the victim intended to submit.

Exploiting the vulnerability: The Brief description field only accepts a single line, so I minified the Q2.3 script onto one line before pasting it in: every `//` comment had to be removed first, since once joined onto one line a comment would silence everything typed after it, and I added one semicolon after `Ajax.open(...)` that the original, one-statement-per-line version relied on JavaScript's automatic semicolon insertion to supply. While logged in as samy, I pasted the resulting one-line script into Brief description and saved, as shown in Figure 2.10. Figure 2.11

shows Samy's own profile immediately afterwards: Brief description renders no visible text, because a `<script>` element produces no displayed output, only execution, consistent with Q2.1 and Q2.2.

To verify the attack from a victim's side, I first checked Alice's profile before any of this ran: Figure 2.12 shows `profile/alice` with no About me or Brief description content at all. I then logged in as alice, opened Firefox's Network tab, and opened the Members listing, the same page that rendered Samy's payload in Q2.1 and Q2.2. Figure 2.13 shows the result: a POST request to `action/profile/edit` fired automatically, with Initiator `members:41 (xhr)`, meaning it was issued by a script running on that page rather than by me submitting any form. Its form data shows `name: "Alice"`, `description: "s4041605"`, and `guid: "56"` – Alice's own name and GUID, supplied automatically by the worm reading her active session, together with the injected student number. 56 also differs from the fixed `samyGuid` of 59 written into the script in Q2.3, so the worm's `if` guard correctly identified Alice as a different user from Samy and proceeded to send the forged request. Figure 2.14 confirms the outcome: Alice's own profile now shows About me as `s4041605`, without Alice ever opening an edit form or clicking Save herself.

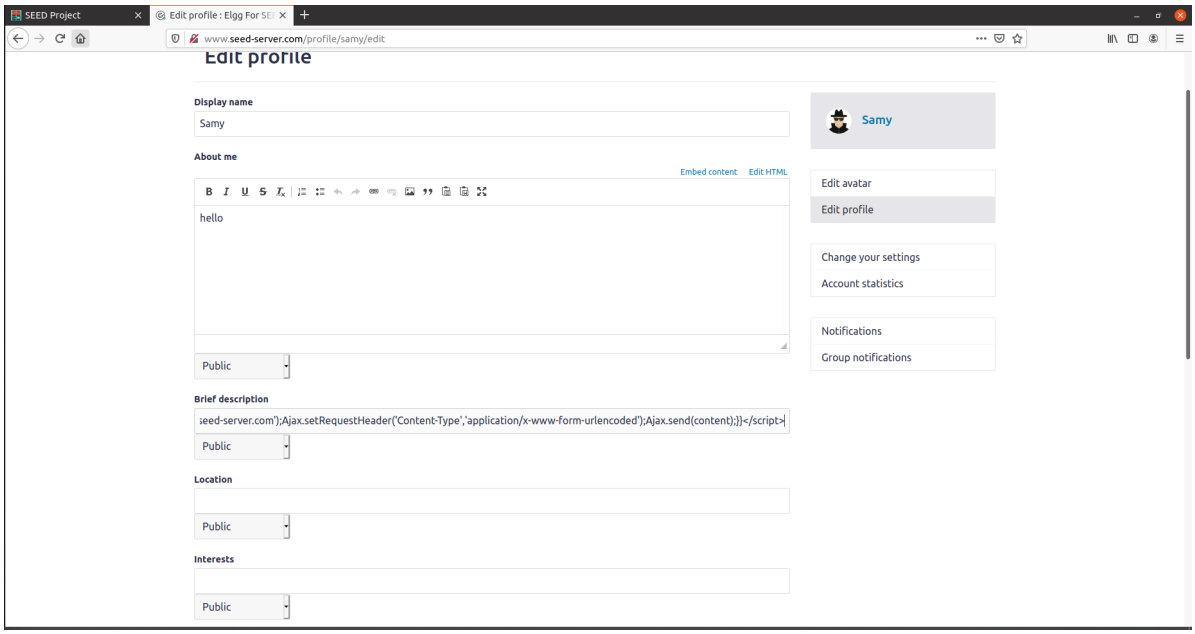


Figure 2.10. The minified worm script saved in Samy's Brief description field.

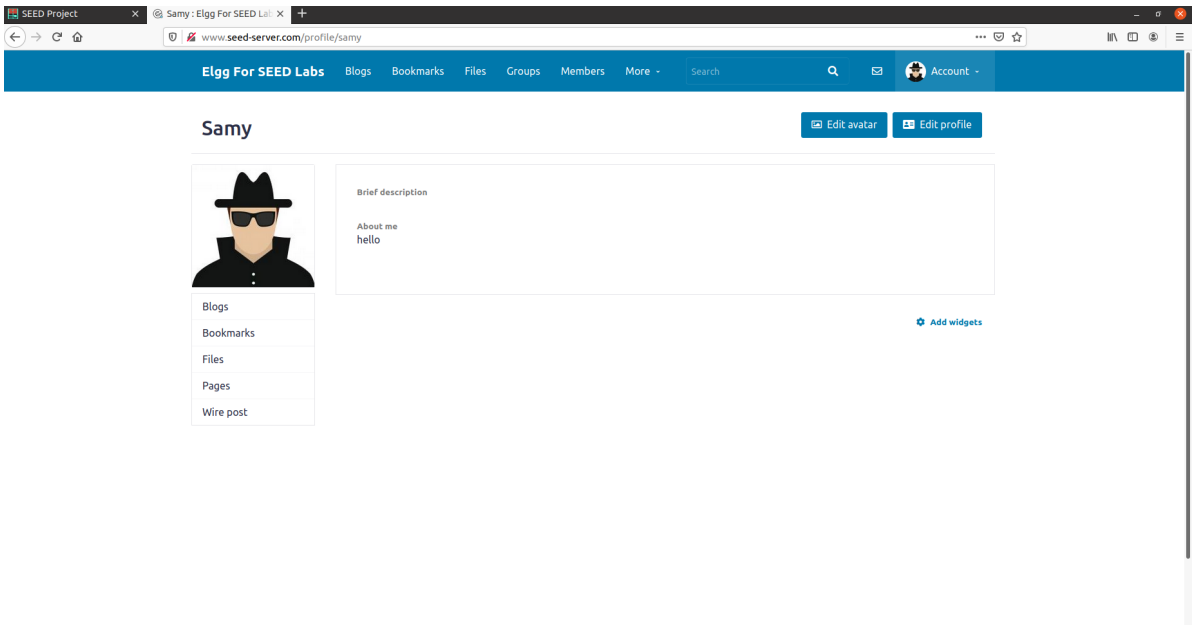


Figure 2.11. Samy's own profile after saving, with Brief description rendering no visible text since it now contains only a script element.

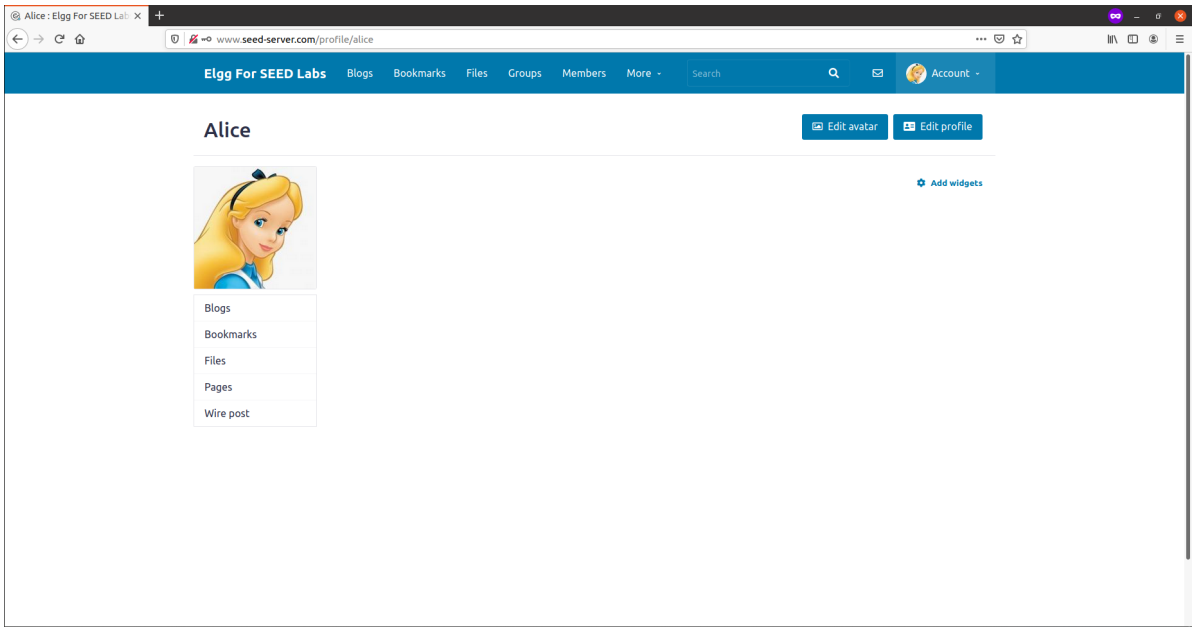


Figure 2.12. Alice's profile before the attack, with no About me content.

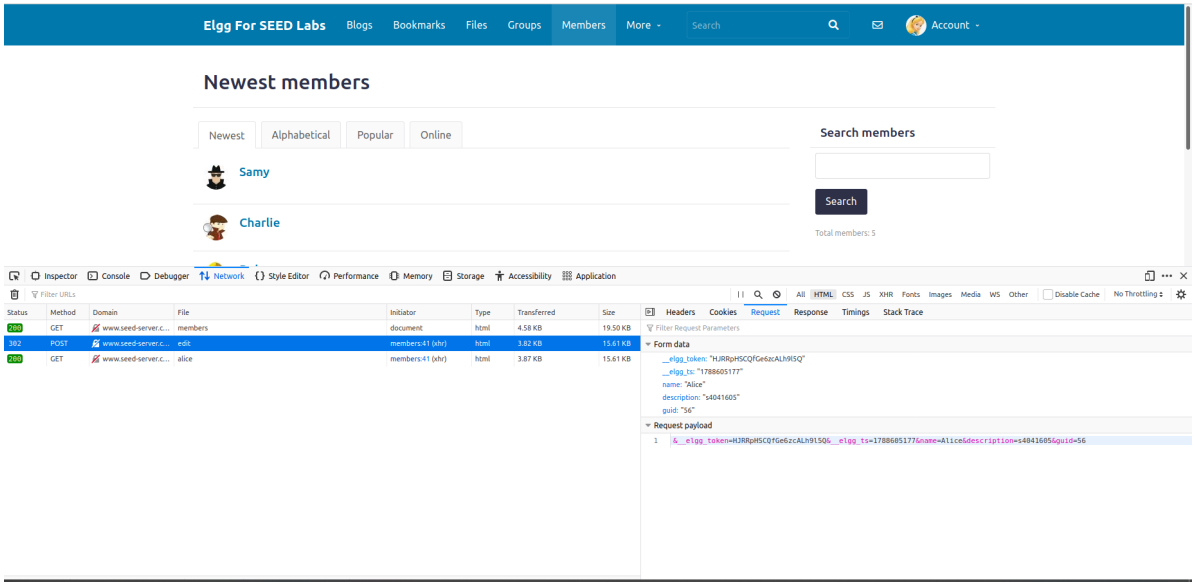


Figure 2.13. The forged POST request firing automatically in Alice's session while she merely opened the Members listing page, with her own name, GUID, and the injected description value visible in its form data.

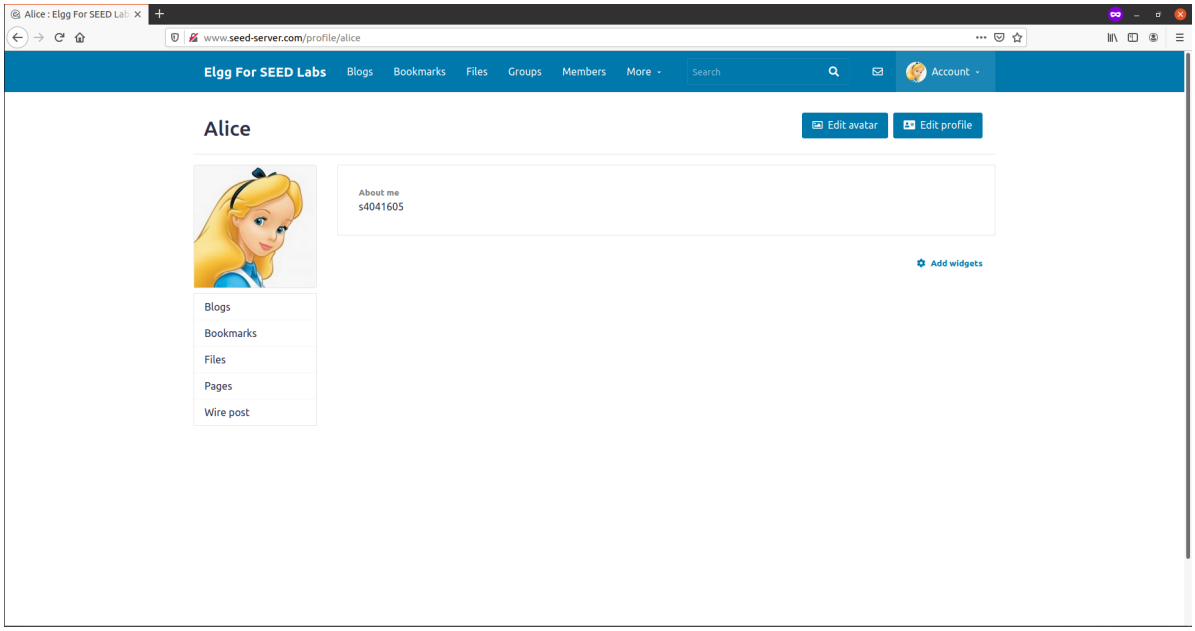


Figure 2.14. Alice's profile after the attack, with About me automatically changed to s4041605.

Conclusion: The evidence demonstrates a working XSS worm: a script stored once in Samy's profile forged an authenticated profile-edit request from inside a different user's own browser session, using that victim's own security token and GUID, and changed her profile without her submitting any input of her own. This differs from the classic self-replicating Samy worm in scope: the version built here writes a fixed value into the victim's About me field and stops, rather than also copying itself into the victim's own profile to keep spreading to further visitors, since the assignment specifies only the former.

3. Analyzing the Security of AI Systems

Q3.1 Provide screenshots to report your observations.

Answer:

Using adversarial.js's MNIST (digit recognition) model, I selected the source image showing the digit 3, set "Turn this image into a" to 8, chose the "Jacobian-based Saliency (stronger)" attack, and generated the adversarial example. Figure 3.1 shows the full result.

The Original Image panel shows the unmodified digit fed to the network: Prediction "3" at Probability 96.09%, marked "Prediction is correct." The Adversarial Image panel shows the output of the attack: the same digit shape, but with a sparse scattering of extra white pixels added near the image's edges and corners that are not present in the original. Feeding this perturbed image to the identical network produced Prediction "8" at Probability 99.86%, with the tool itself reporting "Prediction is wrong. Attack succeeded!"

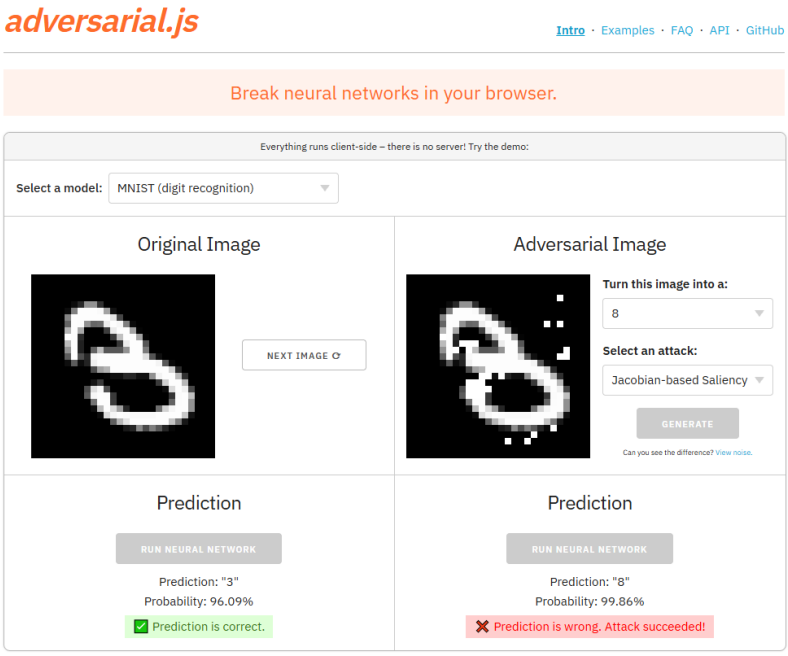


Figure 3.1. The MNIST model's original prediction on the digit 3, and its prediction on the Jacobian-based Saliency (stronger) adversarial example targeting 8.

The two images are visually near-identical to a human observer – the digit's overall loops and strokes are unchanged, and the added pixels are few and confined to the background rather than the digit itself – yet the network's output flipped from 3 to 8 with higher confidence than its original, correct prediction. This demonstrates a successful evasion attack: a small, targeted perturbation caused a confident misclassification without altering what the image depicts to a human viewer.

Q3.2 Analyze the reasons for this successful evasion attack, such as image similarity, referring to the attack patterns on page 19 of the Week 6 lecture.

Answer:

The attack succeeded because it changed very little of the image, and it did not choose that little to change at random. Comparing the two panels of Figure 3.1, the digit's own strokes are untouched; the only visible difference is a handful of extra white pixels scattered in the background near the edges and corners. The two images are, to a human viewer, the same digit 3 with a light speckle of noise, which is why the attack is called an evasion attack rather than a change to the underlying content: the input still depicts a 3, but the network's output changed anyway.

Page 19 of the Week 6 lecture, "Typical Attacks: Evasion Attacks – 4b" (Figure 3.2), illustrates why a perturbation this small is enough. Its left-hand **Gradient-based methods** panel (GradArgmax) shows an evasion attack proceeding in explicit steps: at Step 1, the attacker computes the gradient of the model's output on the clean input, reads off which single component has the largest gradient value (highlighted in red, value 10 in the slide's example), and perturbs only that one component, producing $G + \Delta G_1$. At Step 2, the gradient is recomputed on this new, already-perturbed input, a new highest-value component is identified, and that one is perturbed next. Each step targets whichever single input component the model's own gradient says will move its output furthest toward the attacker's goal, rather than perturbing the input broadly or uniformly.

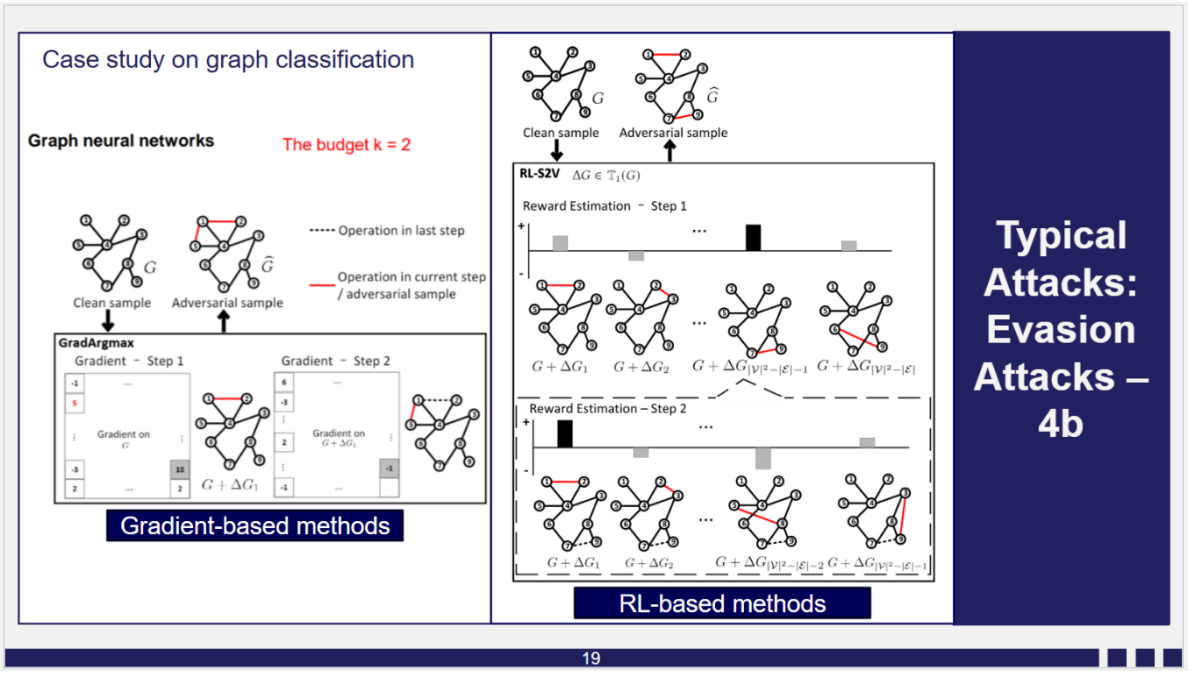


Figure 3.2. Week 6 lecture, page 19: "Typical Attacks: Evasion Attacks – 4b", showing the gradient-based (GradArgmax) and RL-based (RL-S2V) evasion-attack methods for graph neural networks.

The Jacobian-based Saliency Map Attack used in Q3.1 follows the same pattern, applied to image pixels instead of graph edges: it computes the Jacobian of the network's output with respect to every input pixel, which is a matrix of exactly the per-pixel gradients the slide's method reads one value from, ranks pixels by how much increasing them would raise the target class's score,

modifies the highest-ranked pixel or pixels by the maximum allowed amount, and repeats with a freshly computed Jacobian on the updated image, exactly as the slide's Step 1 / Step 2 process recomputes its gradient after each perturbation. Because both methods select their perturbation from the model's own local gradient information rather than from the input's visual content, the resulting change is concentrated on whichever few pixels the network's decision boundary is most sensitive to, which is generally a small subset of all the pixels in the image. This is also why the perturbation lands in the background rather than redrawing the digit itself: those background pixels are the ones the trained network's gradient identifies as most influential for the digit-8 output, not the pixels a human would associate with the number 3.

This explains the observed result without needing to inspect the model's internals directly: the high visual similarity between the original and adversarial images in Figure 3.1 is the direct, observed consequence of the attack modifying only the few pixels its gradient computation ranked highest, and the confident misclassification (99.86% for 8) is the intended outcome of an iterative, gradient-guided search of exactly the kind Page 19 illustrates for a different model type. The lecture's case study is on graph neural networks rather than image classifiers, so the specific inputs being perturbed differ, but the underlying attack pattern – use the model's own gradient to find the input's most sensitive components, perturb only those, and repeat – is the same mechanism in both cases, and it is what makes a visually minor change sufficient to flip the network's prediction.